

Sutton and Barton Exercises Chapter 5

danieltocco0

November 2025

1 Chapter 6 Temporal-Difference Learning

Exercise 6.1 If V changes during the episode, then (6.6) only holds approximately; what would the difference be between the two sides? Let V_t denote the array of state values used at time t in the TD error (6.5) and in the TD update (6.2). Redo the derivation above to determine the additional amount that must be added to the sum of TD errors in order to equal the Monte Carlo error.

Taking $\delta_t + \gamma(G_{t+1} - V(S_{t+1}))$, we know at the next time step $V(S_{t+1})$ will be updated, so $V_t(S_{t+1}) \neq V_{t+1}(S_{t+1})$. To account for this, we modify our original equation so it becomes:

$$\begin{aligned} & \delta_t + \gamma(G_{t+1} - V_t(S_{t+1})) + V_{t+1}(S_{t+1}) - V_{t+1}(S_{t+1}) \\ &= \delta_t + \gamma(G_{t+1} - V_{t+1}(S_{t+1})) + (V_{t+1}(S_{t+1}) - V_t(S_{t+1})). \end{aligned}$$

This results in:

$$G_t - V_t(S_t) = \sum_{k=t}^{T-1} \gamma^{k-t} \delta_k + \sum_{k=t}^{T-1} \delta^{k-t+1} (V_{k+1}(S_{k+1}) - V_k(S_{k+1})).$$

To emphasize the error term is *not* available at $k=t$ because we're using TD(0), hence the sum starts + 1 step ahead. Also note that if $V_{k+1}(S_{k+1}) = V_k(S_{k+1})$ then the error term vanishes.

Exercise 6.2 This is an exercise to help develop your intuition about why TD methods are often more efficient than Monte Carlo methods. Consider the driving home example and how it is addressed by TD and Monte Carlo methods. Can you imagine a scenario in which a TD update would be better on average than a Monte Carlo update? Give an example scenario—a description of past experience and a current state—in which you would expect the TD update to be better. Here's a hint: Suppose you have lots of experience driving home from work. Then you move to a new building and a new parking lot (but you still enter the highway at the same place). Now you are starting to learn predictions

for the new building. Can you see why TD updates are likely to be much better, at least initially, in this case? Might the same sort of thing happen in the original scenario?

The benefits seem similar to the idea of transfer learning, but the downstream tasks are what are being transferred. Consider an orchestra being exceptional at playing Beethoven. Now what if they are asked to play some neo-Beethoven, which begins with some experimental musical elements, and then gradually returns to normal Beethoven. A Monte Carlo algorithm that is tasked with performing this will probably play poorly at first, as it has to wait until the finale to be scored. A TD algorithm will be able to transition more smoothly from neo-Beethoven to normal Beethoven. This is a contrived, and probably not so good example, but I wanted to deviate from the traffic example.

Exercise 6.3 From the results shown in the left graph of the random walk example it appears that the first episode results in a change in only $V(A)$. What does this tell you about what happened on the first episode? Why was only the estimate for this one state changed? By exactly how much was it changed?

That the first step was to the left and terminated. No other states were visited, so there is no sample data to update them with. Using the update rule of 6.2, the update should be $(0.5) + 0.01(0 + 0 - 0.5) = 0.495$.

Exercise 6.4 The specific results shown in the right graph of the random walk example are dependent on the value of the step-size parameter, α . Do you think the conclusions about which algorithm is better would be affected if a wider range of α values were used? Is there a different, fixed value of α at which either algorithm would have performed significantly better than shown? Why or why not?

Based on this section, this is still an open-ended question in terms of proof. In practice, the conclusion would remain the same. TD algorithms have less variance because they only update looking one step ahead, whereas MC methods use the whole reward sequence. Thus, a small α is needed to help contain the variance of MC methods, but this results in slower convergence. On the other hand, TD algorithms can use a larger α because the variance is smaller, but a bias towards $V(S+1)$ is introduced. Perhaps a schedule for α will make the MC algorithm perform closer (if we had infinite resources, we could get it to converge to the true value with $\alpha \rightarrow 0$, whereas TD will still fluctuate due to

the bias).

Exercise 6.5 In the right graph of the random walk example, the RMS error of the TD method seems to go down and then up again, particularly at high α 's. What could have caused this? Do you think this always occurs, or might it be a function of how the approximate value function was initialized?

Again, TD is a bootstrapping method. It bases estimates on other estimates. At first it will reduce error quickly, but eventually fluctuate around a biased fixed point. If α is large, then the fluctuations are more pronounced. The curvature is a function of how the approximate value function was initialized; consider the case where the initialization was very close to the truth, in which case the error is very small. With small initial error, the TD update errors to approximate converge will be smaller, and consequently the fluctuations around the approximate convergent point will be small.

Exercise 6.6 In Example 6.2 we stated that the true values for the random walk example are $\frac{1}{6}, \frac{2}{6}, \frac{3}{6}, \frac{4}{6}$, and $\frac{5}{6}$, for states A through E. Describe at least two different ways that these could have been computed. Which would you guess we actually used? Why?

Way one would be: $\frac{\text{distance-from-left-terminal}}{\text{span-of-terminals}}$, which upon further research is: $v(i) = \frac{1}{2}v(i-1) + \frac{1}{2}v(i+1)$. (A = position 1). The other method would be to use Monte Carlo Simulation with infinite samples (TD will only approximate, as per the above answers). Assuredly, the former was used.

Exercise 6.7 Design an off-policy version of the TD(0) update that can be used with arbitrary target policy π and covering behavior policy b , using at each step t the importance sampling ratio $p_{t:t}$ (5.3).

Originally, I assumed to just update $\rho_{t:t}[R_{t+1} + \gamma V(S_{t+1})]$, which according to GPT, is wrong. Why? What I just wrote is just the target. We need the expected update: $E[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$. The full update with importance sampling ratios then becomes: $V(S_t) \leftarrow V(S_t) + \alpha \rho_{t:t} \delta_t$.

Exercise 6.8 I am going to be extremely lazy because I am sick, use a very loose definition of "show", not write the question, and simply state that the $V(s)$ es can be replaced by $Q(S, A)$ es in 6.6.

Exercise 6.9 I am going to be extremely lazy because I am sick, use a very

loose definition of "show", not write the question, and simply state that the $V(s)$ es can be replaced by $Q(S, A)$ es in 6.6.

Exercise 6.11 Why is Q-learning considered an off-policy control method?

If we look at 6.8, we can see the target is always the greedy action $\gamma \max_a Q(S_{t+1}, a)$. This is even if the policy being followed is non-greedy, such as an ϵ -greedy policy.

Exercise 6.11 Suppose action selection is greedy. Is Q-learning then exactly the same algorithm as Sarsa? Will they make exactly the same action selections and weight updates?

It depends on what is meant by greedy; if Sarsa follows an ϵ -greedy, then no. They may be similar overall, though. If Sarsa follows an ϵ -greedy where $\epsilon = 0$, then yes. They are the same algorithm at that point.

Exercise 6.13 What are the update equations for Double Expected Sarsa with an ϵ -greedy target policy?

$$Q_1(S, A) \leftarrow Q_1(S, A) + \alpha[R + \gamma \sum_{a'} \pi(A'|S') Q_2(S', A') - Q_1(S, A)]$$

$$Q_2(S, A) \leftarrow Q_2(S, A) + \alpha[R + \gamma \sum_{a'} \pi(A'|S') Q_1(S', A') - Q_2(S, A)]$$

Skipping 6.14 – I have to return to programming exercises after first read through of the book.